

BioShield-AI Sentinel: A Convolutional Neural Network Approach for AI-Assisted Biosecurity Screening of Nucleic Acid Synthesis Orders

Roshan George <i>Dept. of Computer Science RV College of Engineering Bengaluru, India roshangeorge.cs24@rvce.edu.in</i>	Mansi Rai <i>Dept. of Computer Science RV College of Engineering Bengaluru, India mansirai.cs24@rvce.edu.in</i>	Pedada Punarv <i>Dept. of Computer Science RV College of Engineering Bengaluru, India pedadapunarv.cs24@rvce.edu.in</i>
	Prof. Ajeet Kumar Srivastava <i>Dept. of Biotechnology RV College of Engineering Bengaluru, India</i>	

Abstract—The growing accessibility of benchtop DNA synthesis technology has introduced significant biosecurity risks, since current synthesis-screening tools rely primarily on static string-matching against curated pathogen sequence databases, an approach that can be circumvented through synonymous mutations or deliberate sequence obfuscation. This paper presents BioShield-AI Sentinel, a prototype deep learning-based system for detecting potentially hazardous nucleic acid sequences. The system decomposes a raw DNA sequence into overlapping 6-mer tokens, learns dense embedding representations for each token, and applies a one-dimensional convolutional neural network (1D-CNN) with adaptive max-pooling to identify local sequence motifs associated with biological risk. The model was trained and evaluated on the NIST Nucleic Acid Synthesis Screening Test Dataset, comprising 1,000 labeled 200-base-pair sequences, and achieved 61.0% accuracy on a held-out, class-balanced test set, returning both a binary verdict and a confidence score for each submitted sequence. An interactive Streamlit-based dashboard was developed to allow DNA synthesis providers to submit candidate sequences for screening and review per-fragment diagnostics. BioShield-AI Sentinel is intended as a complementary screening layer that operates alongside, rather than in place of, existing database-matching approaches, providing additional motif-based detection capability that may help flag sequences modified through obfuscation techniques for further expert review.

Index Terms—biosecurity, DNA synthesis screening, deep learning, convolutional neural network, sequence classification, biological threat detection

I. INTRODUCTION

The accessibility of benchtop DNA synthesis technology has expanded dramatically over the past decade, making it cheaper and easier to produce custom nucleic acid

sequences. While this expansion has accelerated research in synthetic biology, genomics, and medicine, it has also raised a pressing biosecurity concern: the same infrastructure that enables legitimate research could be misused to synthesize sequences associated with hazardous biological agents [10].

In response, synthesis providers and regulatory bodies have established sequence-screening protocols intended to flag dangerous orders before fulfillment. These protocols rely mainly on static database matching, comparing submitted sequences against curated lists of known pathogen-associated sequences [11]. This approach has limitations: sequences can evade detection through codon substitutions, minor mutations, or deliberate obfuscation that preserve biological function while altering the literal nucleotide string. Such challenges have motivated the development of more intelligent computational approaches for sequence analysis [10], [11].

The limitations of traditional pattern-matching approaches in genomics are well documented, and the broader bioinformatics community has increasingly adopted machine learning and deep learning techniques to capture complex biological patterns that static methods may overlook [14], [15]. Convolutional Neural Networks (CNNs) have proven particularly effective when adapted to biological sequence data [3], [8], [13]. CNNs detect position-invariant sequence patterns and local motifs, making them naturally suited to identifying functional regions embedded within long DNA sequences. Several studies have successfully applied CNN-based architectures to DNA sequence classification, regulatory sequence prediction, and DNA-protein binding analysis [3], [6], [8], [13].

Despite the growing body of research applying deep learning to genomic sequence analysis, limited work has focused on integrating these models into practical operational workflows for nucleic acid synthesis

biosecurity screening. This gap motivates the development of the system proposed in this work [5], [15].

We present BioShield-AI Sentinel, a prototype intrusion-detection-style system for nucleic acid synthesis screening. The system tokenizes raw DNA sequences into overlapping 6-mer words, converts them into learned embedding representations, and processes them using a one-dimensional CNN architecture with adaptive max-pooling. This approach is inspired by advances in biological sequence embeddings and deep learning-based genomic analysis [7], [3], [13].

The model is trained using the NIST Nucleic Acid Synthesis Screening Test Dataset, a benchmark dataset designed for evaluating DNA synthesis screening methodologies [12]. A Streamlit-based interactive dashboard was also developed, allowing users to submit candidate DNA sequences and receive a classification verdict together with a confidence-based risk assessment.

II. TOOLS AND TECHNIQUE

A. Development Stack

The entire BioShield-AI Sentinel system is developed in Python. PyTorch is used as the deep learning framework for designing, training, and deploying the DNA sequence classification model; it has become one of the most widely adopted frameworks for implementing modern deep learning architectures [1], [14]. Biopython is used for parsing raw DNA sequences stored in standard biological file formats such as FASTA. Pandas and NumPy are used for data preprocessing, manipulation, and numerical operations. The trained model is stored using PyTorch serialization together with a pickled metadata file, allowing the learned parameters and vocabulary to be reused during inference without retraining. Table I summarizes the tools and libraries used in this study.

B. Sequence Tokenization: *k*-mer Decomposition

Raw DNA sequences cannot be processed directly by neural networks and must first be converted into a machine-readable representation. This is achieved through *k*-mer decomposition, in which each DNA sequence is split into overlapping subsequences of length six (6-mers). This representation preserves local biological information and allows the model to identify meaningful sequence patterns [7]. For example, a DNA sequence of length 10 decomposes into five overlapping 6-mers, where each consecutive *k*-mer shares five nucleotides with its neighbor.

C. Vocabulary Construction and Vectorization

A vocabulary is built from the complete set of unique 6-mers present in the training data. Index 0 is reserved for padding (PAD) and index 1 for unrecognized tokens (UNK); all vocabulary entries are assigned indices starting at 2, so that padding and unknown tokens are never

conflated with a learned token. Each unique *k*-mer is assigned a numerical index, transforming biological sequences into integer-based representations suitable for neural network processing [7]. To ensure a consistent input size, sequences are truncated or zero-padded to a fixed maximum length of 500 tokens before being passed to the network.

D. Learned Embeddings

The model uses an embedding layer to learn dense numerical representations of individual 6-mers. Similar *k*-mers are mapped to nearby locations in the embedding space, enabling the model to capture hidden relationships between biological sequence fragments. This concept is analogous to word embeddings used in natural language processing and has been successfully applied to genomic sequence analysis [7], [15].

E. Convolutional Motif Detection

A one-dimensional Convolutional Neural Network (1D-CNN) layer is used to detect local sequence motifs that may indicate biologically significant regions. CNNs can recognize important patterns regardless of their exact location within a DNA sequence, making them well suited to genomic analysis and motif discovery [3], [8], [13]. For example, if a particular sequence motif is associated with a hazardous biological function, the convolutional filters can learn to activate when similar patterns appear anywhere within the input sequence.

F. Adaptive Max-Pooling and Classification

Following convolution, adaptive max-pooling is applied to retain the strongest activation from each learned feature channel. This highlights the most informative motifs while reducing the dimensionality of the sequence representation. The resulting feature vector is then passed through a fully connected classification layer to predict the final risk category of the DNA sequence [3], [13].

G. Training Procedure

The network is trained using the Adam optimization algorithm, which updates model parameters by minimizing classification loss based on prediction error. Adam is a standard optimization technique for training deep neural networks owing to its efficient adaptive learning-rate strategy [1]. The model is trained on the NIST Nucleic Acid Synthesis Screening Test Dataset, which provides benchmark DNA sequences for evaluating nucleic acid synthesis screening approaches [12].

H. Deployment: Interactive Screening Dashboard

The trained model is integrated into an interactive Streamlit-based dashboard that allows users to perform DNA sequence screening without direct access to the underlying code. Users can submit a DNA sequence, receive a classification prediction, and view a confidence-based analysis of the model's decision. The

dashboard also provides a diagnostic view of detected sequence fragments, supporting interpretability of the screening process. The goal of this deployment is to translate deep learning-based genomic screening from a standalone computational model into a practical biosecurity assistance tool [10], [15].

TABLE I TOOLS AND LIBRARIES USED IN THIS STUDY

Tool / Library	Category
PyTorch	Deep learning framework
Biopython (Bio.SeqIO)	Bioinformatics library
pandas	Data manipulation
NumPy	Numerical computing
pickle	Serialization
Streamlit	Web application framework
Custom k-mer tokenizer (features.py)	Sequence preprocessing
Custom vectorizer (vectorizer.py)	Sequence preprocessing

III. METHODOLOGY

A. System Overview

The proposed system follows a four-stage pipeline. First, raw DNA sequences are matched with their corresponding ground-truth labels from the source metadata. Second, each sequence is decomposed into overlapping k-mers and converted into a fixed-length integer representation. Third, a convolutional neural network is trained on the resulting representations to classify sequences as hazardous or benign. Fourth, the trained model is packaged into a Streamlit-based tool that allows end users to screen new DNA sequences. This workflow is similar to prior CNN-based DNA classification pipelines [6], [13], adapted here for the biosecurity screening use case described in Section I.

B. Dataset

The model was trained on the NIST Nucleic Acid Synthesis Screening Test Dataset [12], which contains 1,000 sequences of 200 base pairs each: 500 true-positive (TP) sequences drawn from BSAT-organism virulence-factor genes and 500 true-negative (TN) sequences drawn from non-pathogenic bacterial and phage genomes. Sequences are distributed as a FASTA file with blinded, sequentially numbered headers, accompanied by a tab-separated metadata file. The metadata file contains a sequential row index column (not a risk label), a Designation column giving the ground-truth classification (TP or TN), and a Name column with a descriptive source identifier. The FASTA sequences and metadata are joined on the shared sequence identifier and the Designation column is used as the classification target, which is encoded as 1 (TP) or 0 (TN) prior to training.

C. Preprocessing Pipeline

Each DNA sequence is processed identically during both training and inference to guarantee a consistent input representation. First, the sequence is decomposed into overlapping 6-mers. Second, a vocabulary is built from the complete set of unique 6-mers observed in the training data, with index 0 reserved for padding and index 1 reserved for unrecognized tokens. Third, each sequence is mapped to a fixed-length integer array of length 500, truncating longer sequences and zero-padding shorter ones. The corresponding label is retained alongside each resulting integer array, producing a tokenized dataset that is passed to the model in Section III-D.

D. Model Architecture

The classification model, BioShieldCNN, consists of four layers. An embedding layer maps each integer token to a learned 64-dimensional dense vector [7], [15]. A one-dimensional convolutional layer with 128 output channels and a kernel size of 3 then scans the embedded sequence to detect local motifs, regardless of their position within the sequence [3], [8], [13]. A ReLU activation is followed by an adaptive global max-pooling layer, which retains the strongest motif activation for each of the 128 channels [3], [13]. Finally, a fully connected linear layer maps the resulting 128-dimensional feature vector to the two output classes (TN, TP). This design was chosen specifically because convolution combined with global max-pooling allows the network to detect a discriminative motif irrespective of where it occurs within the 200 base-pair input, which is essential given that hazardous motifs may appear at arbitrary positions within a submitted sequence [3], [8].

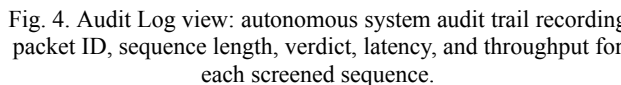
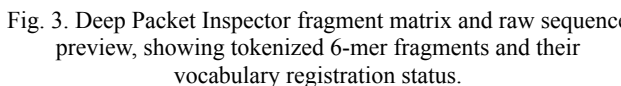
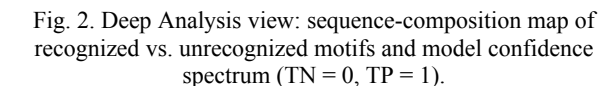
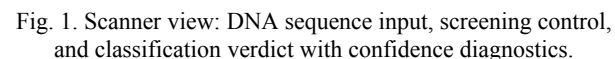
E. Training Procedure

The model was trained for 10 epochs using the Adam optimizer with a learning rate of 0.001 and a cross-entropy loss function [1], [16]. The 1,000-sequence dataset was split into an 80/20 stratified train/test partition (800 training sequences, 200 test sequences), preserving the original class balance in both partitions. The 200 held-out test sequences were excluded from both vocabulary construction and model training, and were used exclusively for the final evaluation reported in Section IV. A fixed random seed was used for the train/test split to support reproducibility.

F. System Integration and Deployment

After training, the model weights and the vocabulary/metadata required for inference were serialized to disk. A Streamlit-based application loads these artifacts and exposes a screening interface: a user submits a DNA sequence, the application tokenizes and vectorizes it using the same preprocessing pipeline described in Section III-C, and the trained model returns a classification verdict together with a confidence score. Because the saved artifacts are self-contained, the deployed tool can operate without access to the original training data or training

Figs. 1-4 illustrate the deployed dashboard across its three primary views. Fig. 1 shows the Scanner view, where a user pastes a raw DNA string and triggers inference; the interface reports the binary verdict (TN/TP), the number of motifs parsed, inference latency, and throughput. Fig. 2 shows the Deep Analysis view, which presents a sequence-composition breakdown of recognized versus unrecognized 6-mer motifs alongside a model confidence spectrum across the TN/TP classes. Fig. 3 shows the Deep Packet Inspector fragment matrix, which lists each tokenized 6-mer fragment together with its vocabulary registration status and integer vector key, plus a raw-sequence preview used for manual inspection. Fig. 4 shows the Audit Log view, which records a running history of screened sequences with their packet identifiers, sequence length, verdict, latency, and throughput, supporting traceability of past screening decisions.



A. Evaluation Setup

B. Classification Results

The classifier achieved an accuracy of 61.0% on the 200-sample held-out test set. Table II summarizes the overall classification performance, Table III reports per-class precision, recall, and F1-score [17], and Table IV gives the corresponding confusion matrix; the per-class metrics in Table III are computed directly from the confusion matrix in Table IV.

Metric	Value
--------	-------

Metric	Value
--------	-------

Test accuracy	61.0%
Training sequences	800
Test sequences	200
Training epochs	10
Optimizer	Adam (lr = 0.001)
Loss function	Cross-entropy
Output classes	2 (0 = TN benign, 1 = TP hazardous)

TABLE III PER-CLASS METRICS — HELD-OUT TEST SET (N = 200)

Class	Precision	Recall	F1 Score	Support
TN — Benign (0)	0.586	0.750	0.658	100
TP — Hazardous (1)	0.653	0.470	0.547	100
Overall Accuracy	—	0.610	—	200

TABLE IV CONFUSION MATRIX

	Predicted TN	Predicted TP
Actual TN	75	25
Actual TP	53	47

C. Model Configuration

The final model configuration is summarized in Table V. The architecture follows a k-mer embedding and 1D-CNN-based sequence classification approach commonly used in genomic deep learning applications [3], [7], [13].

D. Discussion

The model performed modestly better than chance, with a test accuracy of 61.0% on a balanced binary classification task, an improvement over the 50% expected from random guessing. This indicates that the model learned some discriminative signal from the 6-mer motif distributions and was able to identify useful local sequence patterns through the CNN architecture [3], [7], [13]. The margin over chance is nonetheless modest, and these results should be interpreted as a proof of concept rather than evidence of readiness for operational deployment.

The per-class results in Table III reveal a limitation that is particularly important for biosecurity applications: the model is substantially better at identifying benign sequences (recall = 0.750) than at identifying hazardous sequences (recall = 0.470). In other words, the model is more likely to miss a hazardous sequence than to incorrectly flag a benign one. In a screening context, false negatives are considerably more costly than false positives, since an undetected hazardous sequence may proceed to synthesis. A recall of 0.470 on the hazardous class means the model misses roughly half of all hazardous test sequences, which is not adequate for a system used to make autonomous accept/reject decisions. This finding

underscores the high reliability requirements expected of real-world DNA synthesis screening systems [10], [11].

Several factors likely contribute to the model's limited performance. First, the model was trained for only 10 epochs without hyperparameter optimization, regularization, or more advanced architectural choices that are known to improve deep learning performance [1], [14]. Second, while Adam was used as the optimizer, further gains may be achievable through more careful learning-rate scheduling or alternative optimization strategies [16]. Third, the dataset contains only 1,000 sequences, which is a small sample for a deep learning model with on the order of 289,000 trainable parameters, limiting the model's ability to learn generalizable decision boundaries [15]. Finally, the underlying task is intrinsically difficult: the model must infer hazard status purely from local motif content, without reference to known-sequence databases, which is a harder problem than traditional database-matching screening [10], [11].

TABLE V FINAL MODEL CONFIGURATION

Parameter	Value
Vocabulary size	4,122 (4,120 unique 6-mers + PAD + UNK)
Embedding dimension	64
Convolutional layer	1D Conv, 128 channels, kernel size 3
Pooling	Adaptive global max-pool
Classification head	Linear(128 → 2)
Total trainable parameters	~289,000
Batch strategy	Full-batch (800 samples per step)
Max sequence length	500 tokens

V. CONCLUSION

This paper presented BioShield-AI Sentinel, a prototype system that uses deep learning to screen synthetic nucleic acid sequences for biosecurity applications. The system applies a convolutional neural network that analyzes DNA sequences through 6-mer representations and learns biologically relevant sequence patterns to determine whether a sequence is potentially hazardous or benign [3], [7], [13].

The motivation for this work stems from the limitations of traditional database-matching DNA synthesis screening approaches, which may fail to identify hazardous sequences modified through mutation, codon substitution, or other forms of obfuscation [10], [11]. The proposed system was evaluated on the NIST Nucleic Acid Synthesis Screening Test Dataset, comprising 1,000 labeled sequences of 200 base pairs each [12].

After correcting an error in the data-labeling pipeline that had originally used a non-informative row-index column,

the model was trained and evaluated using a held-out test set, achieving an accuracy of 61.0%, above the 50% baseline expected from random guessing on a balanced binary task. Detailed analysis showed that the model performs better at identifying benign sequences than hazardous ones, resulting in a false-negative rate that is not acceptable for a standalone biosecurity screening system.

Although current performance is insufficient for independent deployment, the prototype demonstrates the feasibility of applying deep learning methods to nucleic acid synthesis screening, and can generate a classification result within milliseconds of deployment, illustrating the potential for low-latency sequence analysis [1], [14], [15]. The accompanying Streamlit dashboard enables users to submit DNA sequences and obtain prediction results in an accessible interface. The modular design of the system allows for future improvement through larger datasets, improved training strategies, more advanced architectures, and pretrained genomic foundation models such as DNABERT and the Nucleotide Transformer [16], [18], [19].

This work contributes toward the development of AI-assisted biosecurity systems intended to complement, rather than replace, existing sequence-screening methods. The proposed model can serve as an additional screening layer that flags suspicious sequences for further expert analysis. Further work involving larger datasets, improved model optimization, and more extensive validation is required before such systems can be reliably applied in real-world DNA synthesis screening workflows [10], [11].

ACKNOWLEDGMENT

The authors thank the National Institute of Standards and Technology (NIST) for developing and providing public access to the Nucleic Acid Synthesis Screening Test Dataset used to train and evaluate the model described in this work.

REFERENCES

- [1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097–1105, 2012.
- [3] D. R. Kelley, J. Snoek, and J. L. Rinn, "Basset: Learning the regulatory code of the accessible genome with deep convolutional neural networks," *Genome Research*, vol. 26, no. 7, pp. 990–999, 2016.
- [4] N. C. K. Chowdhury, A. R. Hossain, and M. A. Islam, "Deep learning based DNA sequence classification: A review," *IEEE Access*, vol. 9, pp. 145220–145241, 2021.
- [5] T. N. Tran, B. Vo, and T. P. Hong, "Deep learning for biological sequence classification: A review," *Current Bioinformatics*, vol. 16, no. 6, 2021.
- [6] T. N. K. Nguyen, S. J. Lee, and K. H. Ryu, "DNA sequence classification using convolutional neural network," in *Proc. Int. Conf. Big Data and Smart Computing*, 2016.
- [7] A. Asgari and M. R. K. Mofrad, "Continuous distributed representation of biological sequences for deep proteomics and genomics," *PLOS ONE*, vol. 10, no. 11, 2015.
- [8] J. Zhou and O. G. Troyanskaya, "Predicting effects of noncoding variants with deep learning-based sequence model," *Nature Methods*, vol. 12, pp. 931–934, 2015.
- [9] T. M. Pearce, J. B. R. Sutherland, and M. A. DeJesus, "Machine learning approaches for pathogen detection from genomic data," *Frontiers in Microbiology*, vol. 11, 2020.
- [10] National Academies of Sciences, Engineering, and Medicine, *Biodefense in the Age of Synthetic Biology*. Washington, DC, USA: National Academies Press, 2018.
- [11] U.S. Department of Health and Human Services, "Screening framework guidance for providers of synthetic double-stranded DNA," 2010.
- [12] National Institute of Standards and Technology (NIST), "Nucleic Acid Synthesis Screening Test Dataset," 2024.
- [13] H. Zeng, M. D. Edwards, G. Liu, and D. K. Gifford, "Convolutional neural network architectures for predicting DNA-protein binding," *Bioinformatics*, vol. 32, no. 12, pp. i121–i127, 2016.
- [14] M. Min, J. Lee, and S. Yoon, "Deep learning in bioinformatics," *Briefings in Bioinformatics*, vol. 18, no. 5, pp. 851–869, 2017.
- [15] C. Angermueller, T. Pärnää, L. Parts, and O. Stegle, "Deep learning for computational biology," *Molecular Systems Biology*, vol. 12, no. 7, 2016.
- [16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
- [17] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [18] J. Ji, B. Luo, Y. Zhou, S. Song, J. Ma, and Z. Zhang, "DNABERT: Pre-trained bidirectional encoder representations from transformers model for DNA-language in genome," *Bioinformatics*, vol. 37, no. 15, pp. 2112–2120, 2021.
- [19] H. Dalla-Torre, L. Gonzalez, M. Mendoza-Revilla, N. Carranza, G. Grzywaczewski, A. Oteri, et al., "The Nucleotide Transformer: Building and evaluating robust foundation models for human genomics," *Nature Methods*, vol. 21, pp. 1461–1472, 2024.